

A BUILD-BY-DOING TUTORIAL

FastAPI Zero to Production

From “what is an API?” to a deployed, tested, authenticated backend — one phase at a time.

1

PHASE ONE

Setup & Your First API

Install FastAPI, write your first endpoint, watch the interactive docs appear, and learn the core develop-run-test loop — building a live API on your own machine.

Contents

1.1 Environment Setup	4
Step 1 — Check your Python version	5
Step 2 — Create a project folder	5
Step 3 — Create a virtual environment	5
Step 4 — Install FastAPI and the server	6
Step 5 — Understand the project structure	6
1.2 Your Very First FastAPI App	7
Run it	8
127.0.0.1 and port 8000 — what are those?	9
1.3 Exploring Auto-Generated Documentation	9
What is this and where did it come from?	9
Test your endpoint from the browser	10
1.4 Response Models and Return Types	10
Setting a custom status code	11
Return types as documentation	12
1.5 Path Operation Decorators Deep Dive	12
What is the app object, really?	13
async def vs def — which should you use?	13
Hands-On Exercise 1: Build a "Math API"	14
Checkpoint: At This Point You Can Now...	16
Phase 1 Summary	17

Your Journey

✓	Foundations: What Is an API?	DONE
1	Setup & Your First API	YOU ARE HERE
2	Path Params, Query Params & Body	
3	Pydantic: Data Validation	
4	Database Integration (SQLAlchemy)	
5	Authentication & Authorization	
6	Advanced FastAPI Features	
7	Testing Your FastAPI App	
8	Deployment with Docker	
★	Capstone — TaskFlow API	

Welcome back. In Phase 0 you built the mental model — clients, servers, HTTP, REST, JSON. You never wrote a line of code, and that was deliberate: you can't build a house without first understanding what a wall is. Now the fun begins. By the end of this phase you'll have written a real, running web server, and you'll see FastAPI's single most delightful feature — automatic, interactive documentation — appear as if by magic. Take it slowly, type the code yourself rather than copy-pasting (your fingers learn things your eyes skip), and don't worry if something breaks. Breaking things and fixing them *is* the learning.

1.1 Environment Setup

Before writing code, we need three things: the right Python, an isolated workspace, and the FastAPI libraries. We'll do each carefully — no shortcuts, because a clean setup now prevents a hundred confusing errors later.

Step 1 — Check your Python version

FastAPI needs **Python 3.11 or newer** to use all the modern type-hint features in this tutorial. Open your terminal and check:

BASH / TERMINAL

```
# Ask Python to print its version number
python --version
```

You should see something like `Python 3.11.7` or higher.

WARNING

Warning: On many systems `python` points to an old Python 2, and you need `python3` instead. If `python --version` shows `2.x`, use `python3` everywhere in this tutorial. Try `python3 --version`. If neither gives you 3.11+, install the latest Python from python.org before continuing.

Step 2 — Create a project folder

Make a fresh folder for this project and move into it:

BASH / TERMINAL

```
# Create a new directory for our project
mkdir book-library-api
# Move into that directory
cd book-library-api
```

Step 3 — Create a virtual environment

A **virtual environment** is a private, isolated copy of Python *just for this project*. Think of it as a sealed lunchbox: the packages you install here can't spill into your other projects, and vice-versa. This is the single most important habit in Python development.

BASH / TERMINAL

```
# Create a virtual environment named "venv" inside this folder
python -m venv venv
```

This creates a `venv/` folder. Now **activate** it — this tells your terminal "use the Python and packages inside venv, not the system ones":

BASH / TERMINAL

```
# Mac / Linux:
source venv/bin/activate

# Windows (PowerShell):
venv\Scripts\Activate.ps1

# Windows (cmd.exe):
venv\Scripts\activate.bat
```

After activating, your terminal prompt changes to show `(venv)` at the front — that's your confirmation it worked.

TIP

Tip: You must activate the virtual environment **every time** you open a new terminal to work on this project. If you ever get a "module not found" error for a package you *know* you installed, the first thing to check is: *is my venv activated?* (Look for the `(venv)` prefix.) To leave the environment later, just type `deactivate`.

Step 4 — Install FastAPI and the server

Now install the two packages we need:

BASH / TERMINAL

```
# fastapi = the framework; uvicorn = the server that runs it
pip install "fastapi[standard]" "uvicorn[standard]"
```

- **fastapi** is the framework itself — the tools for defining endpoints, validation, docs.
- **uvicorn** is an **ASGI server** — the program that actually listens for HTTP requests and hands them to your FastAPI app. (FastAPI *defines* your API; uvicorn *runs* it. Recipe vs. oven.)
- The `[standard]` part pulls in helpful optional extras (like the auto-reloader and faster JSON handling).

Verify the install:

BASH / TERMINAL

```
# List installed packages and filter for fastapi
pip list | grep -i fastapi
```

Step 5 — Understand the project structure

For now our project is tiny — just one file. Here's what your folder looks like:

TEXT

```
book-library-api/  
├─ venv/           ← your isolated Python environment (don't touch, don't commit)  
└─ main.py         ← your application code (we create this next)
```

As the tutorial grows, this folder will grow too — `database.py`, `models.py`, a `routers/` folder, tests, a Dockerfile. But every large project starts with one file. Let's create it.

CONCEPT CHECK

Concept Check: Why do we use a virtual environment instead of just `pip install`-ing everything globally? *Answer: Different projects need different versions of the same package. A global install forces every project to share one version, which eventually causes conflicts ("dependency hell"). A virtual environment gives each project its own isolated set, so they never interfere.*

1.2 Your Very First FastAPI App

Create a file called `main.py` in your project folder and type the following. We'll explain every single line immediately after.

PYTHON

```
# Import the FastAPI class – this is the core of the framework.  
from fastapi import FastAPI  
  
# Create an "application" object. This `app` is the central object that  
# holds all your routes and configuration. The name `app` is a convention.  
app = FastAPI()  
  
# This is a "decorator". It tells FastAPI: "when a GET request arrives  
# for the path /hello, run the function directly below me."  
@app.get("/hello")  
def say_hello():  
    # Whatever we return is automatically converted to JSON and sent  
    # back to the client as the response body.  
    return {"message": "Hello, World!"}
```

That's a complete, working web API. Eleven lines (counting comments). Let's unpack it.

Line-by-line breakdown

- `from fastapi import FastAPI` — imports the `FastAPI` class from the library you installed. This class is the engine.

- `app = FastAPI()` — creates an *instance* of that class. `app` now represents your entire application. When you run the server, uvicorn looks for this object.
- `@app.get("/hello")` — a **path operation decorator**. Read it as two parts: `.get` is the **HTTP method** (from Phase 0!), and `"/hello"` is the **path** (the URL after the domain). Together: "handle GET requests to `/hello`."
- `def say_hello():` — the **path operation function** (also called a *route handler*). FastAPI calls this function whenever a matching request arrives. The name can be anything; pick something descriptive.
- `return {"message": "Hello, World!"}` — you return a plain Python dictionary, and FastAPI **automatically serializes it to JSON** (`{"message": "Hello, World!"}`) and sends it with a `200 OK` status. No manual conversion needed — remember this was a key FastAPI advantage over Flask.

Run it

In your terminal (venv activated), run:

BASH / TERMINAL

```
# uvicorn = the server. "main:app" = in the file main.py, find the object named app.
# --reload = automatically restart the server whenever you save a code change.
uvicorn main:app --reload
```

You'll see output ending in something like:

TEXT

```
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Application startup complete.
```

Now open your browser and visit **`http://127.0.0.1:8000/hello`**. You'll see:

JSON

```
{"message": "Hello, World!"}
```

What just happened? You ran a server on your own computer. Your browser acted as a *client* and sent a `GET` request to the path `/hello`. Uvicorn received it, handed it to FastAPI, which matched the path to your `say_hello` function, ran it, took the dict you returned, turned it into JSON, and sent it back with a `200 OK`. That's the entire request/response cycle from Phase 0 — and you just built the server side of it.

TIP

Tip — what `--reload` does and why: The `--reload` flag makes uvicorn watch your files and restart instantly whenever you save. So you can edit `main.py`, hit save, refresh the browser, and see changes immediately — no manual restart. It's wonderful for development but **never use it in production** (it's slower and uses more memory); we'll cover the production way to run servers in Phase 8.

COMMON MISTAKES

Common Mistakes: - `uvicorn: command not found` → your virtual environment isn't activated, or uvicorn didn't install. Re-activate venv and re-run the install. - Running `uvicorn app:main` (backwards) → the format is `filename:object_name`. Your file is `main.py` and the object is `app`, so it's `main:app`. - Visiting `http://127.0.0.1:8000/` (the root) and seeing `{"detail": "Not Found"}` → that's correct! You only defined `/hello`, not `/`. The 404 is FastAPI working properly.

127.0.0.1 and port 8000 — what are those?

`127.0.0.1` is a special IP address that always means "**this same computer**" (its nickname is `localhost`). Port `8000` is like an apartment number at that address — it's the specific "door" your server is listening on. So `http://127.0.0.1:8000` means "talk to a server running right here on my machine, at door 8000." Only you can reach it for now; in Phase 8 we'll make it reachable by the world.

1.3 Exploring Auto-Generated Documentation

This is the moment FastAPI earns its reputation. With the server still running, visit:

`http://127.0.0.1:8000/docs`

You'll see a clean, interactive webpage listing your `/hello` endpoint — and you never wrote a word of documentation. This is **Swagger UI**.

What is this and where did it come from?

FastAPI read your code — the decorator, the path, the function, the return type — and automatically produced a machine-readable description of your entire API following a standard called **OpenAPI** (formerly "Swagger"). That description is then rendered as the interactive page you're looking at.

There are actually three documentation URLs, all free:

URL	What it is	Best for
<code>/docs</code>	Swagger UI — interactive, clickable, lets you <i>test</i> endpoints	Trying out your API by hand
<code>/redoc</code>	ReDoc — a clean, three-panel reference layout	Reading and sharing as documentation
<code>/openapi.json</code>	The raw OpenAPI schema as JSON	Feeding to other tools (code generators, Postman)

Test your endpoint from the browser

In the `/docs` page:

1. Click the **GET /hello** row to expand it.
2. Click the **"Try it out"** button.
3. Click the big blue **"Execute"** button.
4. Scroll down to **Response body** — you'll see `{"message": "Hello, World!"}` and a `200` status code.

You just made a real HTTP request without a terminal or writing any client code. The docs page *is* a client.

TIP

Tip — why this matters so much: On most teams, the frontend developers, mobile developers, and external partners all need to know exactly how your API works. With FastAPI, you hand them one URL (`/docs`) and they can read *and test* every endpoint themselves. It stays perfectly in sync with your code because it's *generated from your code* — documentation can never drift out of date. This single feature saves teams enormous amounts of time.

CONCEPT CHECK

Concept Check: If you add a brand-new endpoint to `main.py` and save, what do you need to do to see it in `/docs` ? Answer: *Nothing but refresh the page. Because `--reload` restarted the server and the docs are generated from your code, the new endpoint appears automatically.*

1.4 Response Models and Return Types

FastAPI automatically converts what you return into JSON. Let's see the range of what you can return. Add these endpoints to `main.py` (keep the existing ones):

PYTHON

```
# Returning a string – becomes a JSON string: "FastAPI is fun!"
@app.get("/string")
def get_string():
    return "FastAPI is fun!"

# Returning a number – becomes JSON: 42
@app.get("/number")
def get_number():
    return 42

# Returning a list of dicts – becomes a JSON array of objects.
@app.get("/list")
def get_list():
    return [{"id": 1, "name": "Dune"}, {"id": 2, "name": "1984"}]
```

Save, then visit `/string`, `/number`, and `/list` in your browser. FastAPI serialized each Python object into the matching JSON type. Dictionaries become JSON objects, lists become arrays, strings become quoted strings, numbers stay numbers — all automatic.

Setting a custom status code

By default a successful response is `200 OK`. But remember from Phase 0 that creating something should return `201 Created`. You set this with the `status_code` parameter on the decorator:

PYTHON

```
# Import the `status` helper for readable status-code names.
from fastapi import status

# Tell FastAPI this endpoint returns 201 Created on success.
@app.post("/items", status_code=status.HTTP_201_CREATED)
def create_item():
    # In a real app we'd save something here; for now we just confirm.
    return {"message": "Item created"}
```

TIP

Tip: You could write `status_code=201` directly, but `status.HTTP_201_CREATED` is self-documenting — anyone reading it instantly knows what 201 means. Prefer the named constants; your future self will thank you.

Return types as documentation

You can also annotate the function's **return type**. This doesn't change behavior much yet, but it improves your docs and your editor's autocomplete, and it's the gateway to the powerful **Pydantic response models** we'll meet in Phase 3:

PYTHON

```
# The "-> dict" tells readers and tools that this returns a dictionary.
@app.get("/typed")
def typed_endpoint() -> dict:
    return {"status": "ok"}
```

1.5 Path Operation Decorators Deep Dive

You've used `@app.get` and `@app.post`. There's one decorator for each HTTP method from Phase 0:

Decorator	HTTP method	Typical use
<code>@app.get(...)</code>	GET	Read data
<code>@app.post(...)</code>	POST	Create new data
<code>@app.put(...)</code>	PUT	Replace a whole record
<code>@app.patch(...)</code>	PATCH	Update part of a record
<code>@app.delete(...)</code>	DELETE	Remove a record

They all work the same way: decorate a function with a path, and that function handles requests for that method + path. Here's a preview of the full set (you'll flesh these out in later phases):

PYTHON

```

@app.get("/books")          # list books
def list_books():
    return []

@app.post("/books")         # create a book
def create_book():
    return {"created": True}

@app.put("/books/1")       # replace book 1
def replace_book():
    return {"replaced": True}

@app.patch("/books/1")     # update part of book 1
def update_book():
    return {"updated": True}

@app.delete("/books/1")    # delete book 1
def delete_book():
    return {"deleted": True}

```

What is the `app` object, really?

`app = FastAPI()` creates one object that represents your whole application. Every decorator (`@app.get`, etc.) *registers* a route onto this object — it's like pinning a note to a board that says "requests matching this method and path go to this function." When uvicorn runs `main:app`, it's this object it serves.

`async def` vs `def` — which should you use?

You can write your route functions two ways:

PYTHON

```

# Regular ("synchronous") function
@app.get("/sync")
def sync_route():
    return {"type": "sync"}

# Asynchronous function
@app.get("/async")
async def async_route():
    return {"type": "async"}

```

Both work perfectly. Here's the simple rule for now:

- Use `async def` when your function uses `await` to call something that supports it (an async database driver, an async HTTP client). This lets the server handle other requests while waiting.
- Use plain `def` otherwise. FastAPI is smart: it runs plain `def` functions in a way that won't block the server, so you lose nothing by keeping things simple.

TIP

Tip: Don't agonize over this yet. If you're not writing `await` inside the function, use plain `def`. We'll use `async def` deliberately in Phase 4 when we talk to databases. The wrong choice here won't break anything — at worst it's slightly less efficient.

COMMON MISTAKES

Common Mistakes: Writing `async def` and then calling a *blocking* library inside it (like a synchronous database call or `time.sleep`) is worse than using plain `def`, because it blocks the whole server. Rule of thumb: `async def` only when you actually `await` something.

CONCEPT CHECK

Concept Check: You want an endpoint that deletes a user and returns no body. Which decorator and which status code from Phase 0 fit best? Answer: `@app.delete(...)` with `status_code=status.HTTP_204_NO_CONTENT` — 204 means "success, nothing to return."

HANDS-ON

Hands-On Exercise 1: Build a "Math API"

Time to build something yourself. Create a new file `math_api.py` and build four endpoints that do arithmetic using **query parameters** (the `?a=5&b=3` part of a URL — we'll formally cover these in Phase 2, but you'll use them here intuitively).

Requirements:

TEXT

```
GET /add?a=5&b=3      → {"result": 8}
GET /subtract?a=10&b=4 → {"result": 6}
GET /multiply?a=3&b=7  → {"result": 21}
GET /divide?a=10&b=2   → {"result": 5.0}
```

Division by zero must return a clean error, not a server crash.

Here is a complete, working solution (try it yourself first, then compare):

PYTHON

```
# Import FastAPI and HTTPException (for returning clean error responses).
from fastapi import FastAPI, HTTPException, status

app = FastAPI()

# By declaring a and b as int, FastAPI reads them from the query string
# (?a=5&b=3) and automatically converts and validates them as integers.
@app.get("/add")
def add(a: int, b: int):
    return {"result": a + b}

@app.get("/subtract")
def subtract(a: int, b: int):
    return {"result": a - b}

@app.get("/multiply")
def multiply(a: int, b: int):
    return {"result": a * b}

@app.get("/divide")
def divide(a: int, b: int):
    # Guard against division by zero BEFORE doing the math.
    if b == 0:
        # Raising HTTPException sends a proper error response to the client
        # instead of crashing the server with a 500 error.
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Cannot divide by zero",
        )
    return {"result": a / b}
```

Run it (note we point uvicorn at the new file):

BASH / TERMINAL

```
# Run the math_api.py file's app object
uvicorn math_api:app --reload
```

Test it. Visit these URLs in your browser (or use [/docs](#)):

TEXT

```
http://127.0.0.1:8000/add?a=5&b=3      → {"result":8}
http://127.0.0.1:8000/divide?a=10&b=2  → {"result":5.0}
http://127.0.0.1:8000/divide?a=10&b=0  → {"detail":"Cannot divide by zero"} (status 400)
```

What just happened? Two pieces of FastAPI magic appeared. First, by typing `a: int, b: int`, you told FastAPI to pull those values out of the query string and *automatically convert them to integers* — and if someone sends `?a=hello`, FastAPI returns a helpful `422` error without you writing any validation. Second, `HTTPException` let you return a clean, intentional error (`400 Bad Request`) instead of letting the program crash. This is the difference between a toy script and a real API: real APIs fail *gracefully*.

COMMON MISTAKES

Common Mistakes: Trying to handle division by zero with a Python `try/except` that returns a plain dict like `{"error": "..."}` with a `200` status. That *looks* fine but lies to the client — the status code says "success" while the body says "error." Always signal failure with the correct status code, and `HTTPException` does exactly that.

TIP

Tip: Test the error case (`?a=10&b=0`) in `/docs` and look at the response: you'll see status `400` and your `detail` message, nicely formatted. Testing the *unhappy path* is a habit that separates careful developers from hopeful ones.

Checkpoint: At This Point You Can Now...

- ☐ Verify your Python version and explain why **3.11+** is required.
- ☐ Create and **activate a virtual environment**, and explain why isolation matters.
- ☐ Install FastAPI and uvicorn, and explain the difference between the two.
- ☐ Write a minimal FastAPI app and explain **every line** (import, `app`, decorator, handler, return).
- ☐ Run the server with `uvicorn main:app --reload` and explain what `--reload` does.
- ☐ Open and use the **interactive docs** at `/docs`, plus know about `/redoc` and `/openapi.json`.
- ☐ Return strings, numbers, dicts, and lists, and set a **custom status code**.
- ☐ Use all five **path operation decorators** and decide between `def` and `async def`.
- ☐ Build a multi-endpoint API and return clean errors with `HTTPException`.

If any box is unchecked, revisit that section — Phase 2 builds directly on running a server and defining endpoints.

Phase 1 Summary

PHASE 1 • KEY TAKEAWAYS

In this phase you went from nothing to a running, self-documenting API.

- A **virtual environment** isolates your project's packages; activate it every session (look for `(venv)`).
- **FastAPI** is the framework; **uvicorn** is the server that runs it. You start it with `uvicorn main:app --reload`.
- A minimal app is just: import `FastAPI`, create `app = FastAPI()`, decorate a function with `@app.get("/path")`, and `return` a Python object — which FastAPI **auto-converts to JSON**.
- FastAPI **generates interactive docs for free** at `/docs` (Swagger UI), `/redoc`, and `/openapi.json`, all from your code.
- You can set custom status codes with `status_code=...`, and there's a decorator for every HTTP method.
- Type-hinting parameters (`a: int`) gives you **automatic conversion and validation**; `HTTPException` gives you **clean, correct error responses**.
- **You built and tested a 4-endpoint Math API that handles division by zero gracefully.**

You now have the core loop of FastAPI development: write a function, decorate it, run the server, test in `/docs`. Next phase, you'll make endpoints dynamic with path parameters, query parameters, and request bodies.

*End of Phase 1. Reply "**continue**" for Phase 2 — Path Params, Query Params & Request Body — where your endpoints start accepting real input.*